```
# Practical 9: DBSCAN Clustering

# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

# 2. Load the dataset
data = load_iris(as_frame=True)

X = data.data

print("Dataset: Iris Dataset")
print("Number of records:", X.shape[0])
print("Number of features:", X.shape[1])

print("\nFirst five rows:")
print(X.head())

# 3. Scale the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Apply DBSCAN clustering
dbscan = DBSCAN(
    eps=0.8,
    min_samples=5
```

```
)

clusters = dbscan.fit_predict(X_scaled)

# 5. Add cluster labels to dataset
df = X.copy()
df["Cluster"] = clusters

print("\nClustered Data:")
print(df.head(10))

# 6. Count clusters and noise points
unique_labels = set(clusters)

n_clusters = len(unique_labels - {-1})
n_noise = list(clusters).count(-1)

print("\nNumber of Clusters:", n_clusters)
print("Number of Noise Points:", n_noise)

# 7. Calculate Silhouette Score
if 1 < n_clusters < len(X):
    score = silhouette_score(X_scaled, clusters)
    print("Silhouette Score:", round(score, 4))
else:
    print("Silhouette Score cannot be calculated.")

# 8. Visualize clusters
plt.figure(figsize=(8, 5))

plt.scatter(
```

```
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=clusters,
    cmap="viridis",
    s=50
)

plt.xlabel("Feature 1 (Scaled)")
plt.ylabel("Feature 2 (Scaled)")
plt.title("DBSCAN Clustering")
plt.colorbar(label="Cluster")
plt.grid(True)
plt.show()

# 9. Visualize cluster distribution
cluster_counts = pd.Series(clusters).value_counts().sort_index()

print("\nCluster Distribution:")
print(cluster_counts)

plt.figure(figsize=(8, 5))

cluster_counts.plot(kind="bar")

plt.xlabel("Cluster Label")
plt.ylabel("Number of Data Points")
plt.title("DBSCAN Cluster Distribution")
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```